

DAIMO: un perfil de integración para activos de modelos de inteligencia artificial gobernados en espacios de datos

Edmundo de Elvira Mori Orrillo*

Jiayun Liu

Departamento de Inteligencia Artificial, ETSI Informáticos

Universidad Politécnica de Madrid, España

Resumen

Resumen. El despliegue interorganizacional de modelos de inteligencia artificial exige un tratamiento semántico coherente de cinco tareas hoy fragmentadas entre vocabularios distintos: publicar el modelo como activo de catálogo, seleccionarlo bajo una política de uso, invocarlo mediante un servicio tipado, trazar cada ejecución y compararlo sobre condiciones de evaluación reproducibles. MLDCAT-AP describe el modelo, DCAT lo publica, ODRL expresa la política, PROV-O registra la procedencia y el *Dataspace Protocol* (DSP) negocia el intercambio contractual; ninguno articula las cinco tareas como cadena única.

Este artículo presenta DAIMO (*Data-space AI Model Ontology*), un perfil de integración OWL 2 DL que reutiliza los cinco vocabularios citados mediante alineamientos axiomáticos verificados. DAIMO introduce catorce clases nativas (nueve de primer nivel más cinco subclases de rol) que cubren los constructos ausentes en la unión de los vocabularios reutilizados, y se acompaña de una capa SHACL con restricciones estructurales, seis invariantes SHACL-SPARQL entre clases y un banco de pruebas negativas que confirma su carácter discriminante.

DAIMO se desarrolla siguiendo la metodología *Linked Open Terms* (LOT) y se ejerce sobre un caso de estudio multi-participante que instancia las veintitrés preguntas de competencia formuladas durante la especificación de requisitos. El paquete público incluye la ontología, la capa SHACL, el grafo de demostración, las consultas SPARQL y cuatro scripts que reproducen íntegramente los reportes de validación.

Palabras clave: gobernanza de modelos de IA; espacios de datos; alineamiento ontológico; validación SHACL; metodología LOT

1. Introducción

El Reglamento (UE) 2024/1689, conocido como *AI Act*, exige para los sistemas de inteligencia artificial de alto riesgo documentación técnica del modelo, registro de ejecuciones, trazabilidad del entrenamiento y evidencia de auditoría sostenida a lo largo del ciclo de vida [35]. Paralelamente, la iniciativa europea de espacios de datos, materializada en los conectores Eclipse Dataspace Components (EDC) [33] y en despliegues sectoriales como INESData o Gaia-X, consolida un modelo de intercambio contractual entre participantes autónomos sobre el *Dataspace Protocol* (DSP) [32]. La conjunción de ambos marcos sitúa a los modelos de inteligencia artificial en un nuevo régimen: deben poder circular entre organizaciones como activos gobernados, con metadatos accionables que sostengan publicar, descubrir, invocar, trazar y comparar de forma coherente. Los vocabularios maduros de la Web Semántica cubren cada una de esas cinco tareas

*Autor de correspondencia: `edmundo.mori.orrillo@upm.es`.

por separado, y este artículo argumenta que su integración en un único perfil es una contribución tanto necesaria como no trivial.

MLDCAT-AP 3.0.0, publicada por SEMIC en 2025 [31], extiende DCAT para describir modelos de aprendizaje automático como activos de catálogo, con metadatos sobre tarea, algoritmo, ejecuciones, evaluaciones, riesgos y recursos computacionales. DCAT [2] aporta las primitivas de catálogo (`Catalog`, `Dataset`, `Distribution`, `DataService`, `CatalogRecord`) sobre las que MLDCAT-AP se apoya. ODRL [4] proporciona un modelo consolidado de ofertas, permisos, prohibiciones y acuerdos, y es la base natural para expresar las políticas de uso que gobiernan la invocación del modelo. PROV-O [3] es el estándar para procedencia y atribución de entidades, actividades y agentes. DSP [32] especifica los mensajes de negociación de contratos y transferencia que EDC implementa. Las *model cards* [6], *factsheets* [7] y *datasheets* [19] complementan estos vocabularios con descripciones narrativas del uso previsto, las limitaciones y las condiciones de entrenamiento.

Cada uno de estos vocabularios resuelve bien su parte, pero los huecos entre ellos son precisamente los que un perfil de integración debe cerrar. Un modelo publicado como `it6:MachineLearningModel` de MLDCAT-AP no lleva adherido el acto de ser ofrecido a través de una frontera organizativa. Una `odrl:Offer` no está anclada a un modelo concreto dentro de un catálogo federado. Una negociación DSP es agnóstica respecto al tipo de activo negociado y no conoce que el intercambio versa sobre un modelo de IA con métricas y contextos de evaluación que condicionan su uso. Un `prov:Bundle` de PROV-O no agrupa por sí solo las actividades ocurridas bajo contextos administrativos distintos en torno a un mismo artefacto derivado. Las cinco tareas operativas del intercambio exigen articular estos huecos de forma coherente.

No existe todavía una ontología reutilizable que vincule metadatos de publicación en catálogo, políticas de acceso, contratos de invocación, trazas de ejecución, evidencia de auditoría y evaluación comparativa para modelos de IA que circulan entre participantes de un espacio de datos, con alineamientos axiomáticos verificados contra los vocabularios existentes y con evidencia ejecutable y reproducible de su corrección. Cerrar esa distancia exige dos principios simultáneos: reutilizar lo que los vocabularios maduros ya resuelven y añadir sólo los constructos que la articulación entre ellos requiere.

Este artículo presenta DAIMO (*Data-space AI Model Ontology*) y aporta cuatro contribuciones.

La **primera** es la propia ontología de integración: un perfil OWL 2 DL que reutiliza DCAT-AP, MLDCAT-AP 3.0.0, ODRL 2.2, PROV-O y DSP mediante alineamientos axiomáticos `rdfs:subClassOf` y `rdfs:subPropertyOf`, no simples listas de prefijos. Sobre esa capa de reutilización, DAIMO introduce catorce clases nativas que cubren los constructos ausentes en la unión de los vocabularios reutilizados. Cada clase nativa se justifica por un hueco concreto y por al menos una pregunta de competencia que no puede responderse sin ella.

La **segunda** es un mecanismo de *verificación de implicación* sobre el cierre OWL-RL que complementa al razonamiento estándar de consistencia. Para cada clase nativa, la comprobación enumera las superclases inferidas y las contrasta con una lista de superclases prohibidas semánticamente. Este mecanismo detecta alineamientos que un razonador DL acepta sin contradicción pero que introducen tipados no intencionados bajo razonamiento.

La **tercera** es una capa SHACL que integra seis invariantes SPARQL entre clases con un banco de pruebas negativas. Cada invariante dispone de un caso diseñado para violarla. La validación conforma sobre el grafo positivo y dispara las seis violaciones sobre el grafo negativo, lo que demuestra que las restricciones son discriminantes y no meramente satisfacibles.

La **cuarta** es un caso de estudio multi-participante que instancia la ontología sobre un escenario de predicción de riesgo de inundación entre una universidad, un ayuntamiento, un evaluador científico y una plataforma federada de espacios de datos. El caso actúa como demostrador: ejerce cada clase nativa al menos una vez y hace que las veintitrés consultas de competencia devuelvan las filas esperadas.

2. Trabajo relacionado

DAIMO se sitúa en la intersección de cuatro familias de trabajo que la literatura existente trata habitualmente por separado: documentación narrativa de modelos, ontologías del ciclo de vida de aprendizaje automático, vocabularios de gobernanza y estándares de espacios de datos. Revisamos cada familia por orden de proximidad al problema, identificando qué aporta al puente catálogo-dataspace, dónde lo deja abierto y cómo DAIMO se posiciona respecto a ella. Una quinta subsección revisa las ontologías SWJ recientes cuya metodología de validación hemos adoptado. La Tabla 1, inspirada en la matriz de reuso de RePlanIT [9], compara todas las familias contra los cinco atributos operacionales que un perfil para modelos de IA gobernados debe cubrir.

2.1. Descripción de modelos de IA

Documentación narrativa. Las *model cards* de Mitchell et al. [6], las *factsheets* de Arnold et al. [7] y las *datasheets for datasets* de Gebru et al. [19] elevaron los estándares de transparencia sobre artefactos de IA. Describen en lenguaje natural el uso previsto del modelo, sus limitaciones, el rendimiento esperado sobre benchmarks conocidos y las condiciones bajo las que ha sido entrenado y evaluado. Son valiosas para la comunicación con lectores humanos y para la rendición de cuentas cualitativa, pero insuficientes como sustrato para descubrimiento automatizado, filtrado por política o vinculación de evidencia en flujos máquina a máquina: un espacio de datos requiere primitivas que los agentes de software puedan consumir, no documentos que otros agentes deban interpretar.

Ontologías del ciclo de vida ML. Allí donde la documentación narrativa falla en el consumo máquina, las ontologías estructuradas del ciclo de vida ML ocupan su lugar. EXPO [22] ofrece una ontología general de experimentos científicos que ha servido de base a trabajos posteriores. ML-Schema [23] formaliza la semántica de experimentos de aprendizaje automático mediante un esquema reutilizable y ligero. MEX [24] provee un vocabulario de intercambio para resultados de experimentos, y OntoDM [25] junto con DMOP [26] articulan el dominio de la minería de datos y el *semantic meta-mining*. Souza et al. [27] y Schmidt et al. [28] extienden la perspectiva hacia la procedencia y la FAIRness del ciclo de vida ML. Estas ontologías proporcionan un fundamento riguroso pero permanecen centradas en el experimento científico y en el proceso de entrenamiento; ninguna aborda la dimensión contractual del intercambio entre participantes autónomos.

El trabajo más próximo al dominio de DAIMO, y el más reciente, es MLDCAT-AP 3.0.0 [31], publicado por SEMIC en 2025. MLDCAT-AP extiende DCAT para describir modelos de IA como subclases de `dcat:Dataset`, con metadatos ricos sobre política, benchmark, evaluación, ejecución, riesgo e infraestructura computacional. DAIMO lo reutiliza directamente: la clase `it6:MachineLearningModel` es la clase del modelo en DAIMO, y las clases `it6:Task`, `it6:Run`, `it6:Flow`, `it6:Evaluation` e `it6:ComputerInfrastructure` capturan respectivamente la tarea objetivo, la ejecución concreta, el *pipeline* asociado, la medición resultante y el entorno de cómputo. MLDCAT-AP es fuerte en el lado del modelo pero deliberadamente agnóstico respecto al espacio de datos en el que el modelo circula: no modela el acto de ofrecer un modelo a través de fronteras organizativas, ni las nociones de despliegue gobernado, autorización de ejecución anclada a un acuerdo o procedencia agregada entre contextos administrativos distintos. MLDCAT-AP cubre el modelo; DAIMO cubre el puente entre el modelo y el dataspace.

[PENDIENTE: Falta una comparación estructural clase-a-clase con MLDCAT-AP 3.0.0 al estilo de la que GloSIS hace con SOTER, ISO 28258, ANZSoilML e INSPIRE (§2.1–§2.5 de GloSIS). Incluir una tabla o subsección que liste, para cada clase relevante de MLDCAT-AP (MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, HarmRisk,

Modality), si DAIMO la reutiliza directamente, la extiende o la complementa, y en qué dirección aporta.]

2.2. Gobernanza de recursos y espacios de datos

Vocabularios W3C de gobernanza. Las ontologías del ciclo de vida describen qué es un modelo y cómo se produjo, pero no gobiernan cómo se publica, se licencia o se atribuye. Tres vocabularios W3C proporcionan ese sustrato de gobernanza. DCAT 2 [2] proporciona las primitivas de catálogo sobre las que se apoyan tanto MLDCAT-AP como DAIMO. ODRL 2.2 [4] ofrece un modelo consolidado para expresar permisos, prohibiciones, obligaciones, ofertas (`odrl:Offer`) y acuerdos (`odrl:Agreement`) vinculantes; DAIMO lo usa intensivamente tanto para la política adherida a las ofertas como para los acuerdos que autorizan ejecuciones concretas. PROV-O [3] aporta la ontología de procedencia con actividades, entidades, agentes, bundles y roles, y es la capa natural sobre la que codificar la trazabilidad de ejecuciones individuales y su agregación entre participantes. Pandit y Esteves [15] muestran cómo ODRL se combina con DPV para escenarios de salud, un patrón que DAIMO transpone al dominio IA. Estos vocabularios son maduros y ampliamente adoptados, pero por construcción son agnósticos al dominio: ninguno es específico de inteligencia artificial ni de espacios de datos.

Estándares de espacios de datos. Mientras que los vocabularios anteriores describen artefactos, los estándares de espacios de datos describen el intercambio en sí. La perspectiva desplaza el foco desde los artefactos aislados hacia los intercambios gobernados entre actores autónomos. Franklin et al. [1] formularon las *dataspaces* como respuesta a la heterogeneidad persistente de fuentes. Otto et al. [17] y Cappiello et al. [18] convierten la soberanía del dato, los contratos y la interoperabilidad controlada en supuestos explícitos del diseño. El *Dataspace Protocol* [32] es la especificación W3C-CG que define los mensajes de catálogo, la negociación de contratos y los procesos de transferencia; Eclipse Dataspace Components [33] es su implementación de referencia.

Conviene distinguir entre *framework* y *vocabulario*: EDC es un framework de ejecución construido en Java, no una ontología. La semántica que EDC implementa proviene de DSP (protocolo), ODRL (políticas) y DCAT (catálogo); sólo un puñado de conceptos son verdaderas extensiones específicas de EDC, y de ellos DAIMO referencia explícitamente `edc:ParticipantContext` por su utilidad para anclar la procedencia agregada a un contexto administrativo. Esta aclaración es importante porque en la literatura de espacios de datos las dos capas se confunden con frecuencia. Trabajos sobre ecosistemas federados como ConSolid [8] muestran además que la Web Semántica evoluciona hacia escenarios multi-actor donde gobernanza e interoperabilidad deben diseñarse conjuntamente.

2.3. Ingeniería ontológica ejecutable en SWJ

Más allá del sustrato de dominio que las cuatro familias anteriores proporcionan, un quinto hilo de trabajo reciente informa cómo debe evaluarse DAIMO. El *Semantic Web Journal* ha publicado recientemente un conjunto de ontologías que comparten un compromiso metodológico explícito con la validación ejecutable y la reproducibilidad. Kurteva et al. presentan RePlanIT [9], una ontología para pasaportes digitales de producto de equipos ICT, con SHACL y evaluación de usabilidad con expertos de industria. Palma et al. operacionalizan información de suelos a escala global en GloSIS [10] mediante un perfil modular alineado con SOSA/SSN e ISO 28258. Woods et al. conectan ingeniería ontológica con procesamiento del lenguaje natural para actividades de mantenimiento industrial [11]. Bareedu et al. derivan reglas SHACL desde estándares industriales OPC UA [13]. Ferranti et al. formalizan restricciones de propiedad de Wikidata mediante SHACL y SPARQL [14]. Robaldo y Batsakis [16] analizan la interacción

Tabla 1: Matriz de reuso: familias de trabajo relacionado contra los cinco atributos operacionales que un perfil para modelos de IA gobernados debe cubrir. • = cubierto nativamente; ◦ = cubierto parcialmente mediante extensiones; — = no cubierto.

Familia / artefacto	Publicación	Política	Invocación	Trazabilidad	Evaluación
Model cards, factsheets, datasheets [6, 7, 19]	◦	—	—	—	◦
ML-Schema, MEX, OntoDM, DMOP [23, 24, 25, 26]	—	—	—	◦	•
MLDCAT-AP 3.0.0 [31]	•	◦	—	•	•
DCAT 2 [2]	•	—	◦	—	—
ODRL 2.2 [4]	—	•	—	—	—
PROV-O [3]	—	—	—	•	◦
DSP / EDC [32, 33]	◦	◦	•	—	—
DAIMO (este trabajo)	•	•	•	•	•

entre validación SHACL e inferencia sobre la Ontología del Tiempo. Penteado et al. [12] estudian metodologías de publicación de linked open government data. Estos trabajos establecen colectivamente la expectativa —que DAIMO adopta como vinculante— de que una ontología con pretensiones operacionales presente evidencia reproducible de su corrección, no solamente una descripción axiomática.

La lectura horizontal de la Tabla 1 indica que ningún artefacto individual cubre simultáneamente las cinco dimensiones; la lectura vertical confirma que cada columna está cubierta nativamente por al menos un vocabulario reutilizable. DAIMO se diseña, por tanto, como un perfil de integración que ata esas cinco capacidades a través de alineamientos axiomáticos verificados, sin duplicar lo que los vocabularios maduros ya resuelven.

3. Metodología y requisitos

DAIMO adopta la metodología *Linked Open Terms* (LOT) [29] por dos razones. Primera, su orientación industrial: LOT prescribe un flujo ligero con entregables en cada una de sus cuatro fases (especificación de requisitos, implementación, publicación y mantenimiento). Segunda, LOT trata la reutilización como fase de primera clase, no como subproducto, lo que encaja con la regla reuse-first que el problema impone. Las subsecciones siguientes describen el escenario ejecutivo que guió la elicitación de requisitos (§3.1), las preguntas de competencia y los requisitos no funcionales resultantes (§3.2), y las tres decisiones centrales de diseño que dieron forma a la ontología (§3.3).

3.1. Escenario ejecutivo

[PENDIENTE: El escenario descrito a continuación es ilustrativo y todavía no corresponde a un piloto real en curso. Las instituciones nombradas (UPM, Ayuntamiento de Leganés, CSIC, INESData, Gaia-X) son reales, pero la colaboración concreta alrededor del modelo de predicción de inundaciones que se plantea no se ha materializado. Antes del envío a SWJ debe sustituirse por (a) un piloto real documentado con carta de compromiso de las instituciones nombradas, (b) una instanciación sobre un catálogo MLDCAT-AP operativo y público, o (c) una versión anonimizada con actores genéricos (“Proveedor A”, “Consumidor municipal B”, “Plataforma federada C”, “Evaluador científico D”, “Oficina de cumplimiento E”) que no comprometa a instituciones sin su consentimiento. Todas las referencias específicas del cuerpo (§5.1 caso de estudio) —identificadores de modelo, métricas, datasets, endpoints, acuerdos, fechas, hashes— deben actualizarse coherentemente con la opción elegida.]

Tabla 2: Actores involucrados en el escenario ejecutivo y categorías de preguntas de competencia derivadas.

Actor	Categoría	Necesidad	CQs
Proveedor de modelos	R	Publicar un modelo como activo gobernado con identidad, licencia, política y contrato de invocación	CQ-R1–CQ-R5
Consumidor de modelos	D	Descubrir modelos por tarea, dominio, política y umbral de calidad	CQ-D1–CQ-D4
Operador de plataforma	E	Invocar servicios, trazar ejecuciones y reconstruir evidencia operativa	CQ-E1–CQ-E5
Evaluador	V	Comparar resultados en un contexto compartido e inspeccionar reproducibilidad	CQ-V1–CQ-V5
Gobernanza (multi-actor)	G	Puentear oferta, despliegue, autorización y procedencia entre participantes	CQ-G1–CQ-G4

La especificación de requisitos parte de un escenario ejecutivo que posteriormente actúa como caso de estudio (§5.1). La Universidad Politécnica de Madrid (UPM), como proveedor de modelos, publica un modelo de predicción de riesgo de inundación a veinticuatro horas y cien metros de resolución sobre cuencas ibéricas en un catálogo federado gestionado por la plataforma INESData. El Ayuntamiento de Leganés, como consumidor, necesita invocar el modelo para emitir avisos locales tras una tormenta extrema, pero no puede hacerlo sin antes descubrirlo bajo una política compatible con el uso público, verificar que satisface un umbral mínimo de calidad sobre un contexto de evaluación común, negociar un acuerdo que autorice sus ejecuciones concretas e invocarlo a través de un endpoint autenticado. El equipo climático del CSIC, en un rol de evaluador, compara el modelo con una línea base sobre el mismo contexto de evaluación. Una oficina de cumplimiento alineada con Gaia-X, como actor de gobernanza, audita las ejecuciones y archiva la evidencia correspondiente.

Este escenario sintetiza las cinco tareas operativas que una ontología del puente catálogo-dataspace debe soportar simultáneamente (publicación, descubrimiento, invocación, trazabilidad y evaluación) e involucra a cinco tipos de actor claramente diferenciados cuyas necesidades son complementarias pero no coinciden. Es verosímil: no depende de tecnología hipotética y se puede instanciar con los vocabularios reutilizados más las extensiones que DAIMO introduce.

3.2. Especificación de requisitos

Los requisitos se elicitaron desde tres fuentes complementarias: el escenario anterior, el dossier interno de MLDCAT-AP 3.0.0 y el análisis de las extensiones Eclipse EDC empleadas por INESData. A partir de ellas se formularon veintitrés preguntas de competencia, organizadas en cinco categorías por actor y etapa del ciclo de vida. La categoría R (registro y publicación, cinco preguntas) cubre las necesidades del proveedor al publicar el modelo como activo gobernado con identidad, licencia, política y contrato de invocación. La categoría D (descubrimiento y selección, cuatro preguntas) recoge las preguntas que un consumidor plantea para localizar un modelo adecuado por tarea, dominio, política y umbral de calidad. La categoría E (ejecución y auditable, cinco preguntas) agrupa las preguntas que un operador y un actor de gobernanza se plantean para invocar servicios, trazar ejecuciones y reconstruir evidencia. La categoría V (evaluación y reproducibilidad, cinco preguntas) contiene las preguntas del evaluador para comparar resultados bajo condiciones homogéneas. Finalmente, la categoría G (puente de gobernanza, cuatro preguntas) cubre las preguntas multi-actor cuya respuesta exige exclusivamente semántica nativa de DAIMO y que, como se argumenta en §6, no pueden responderse con los vocabularios reutilizados aisladamente. La Tabla 2 resume la correspondencia actor-categoría; el conjunto completo de las veintitrés preguntas en lenguaje natural aparece en el Apéndice A.

Once de las veintitrés preguntas exigen razonamiento explícito (rutas de subclase, cadenas de subpropiedad o agregación SPARQL), mientras que las doce restantes son consultas directas sobre datos DAIMO-nativos. Esta proporción es deliberada: un perfil que se apoyara exclusivamente en consultas directas no ejercitaría la parte del artefacto que hace valiosa la semántica OWL, y un perfil que exigiera razonamiento para toda pregunta sería impracticable en operación sobre endpoints reales. Los requisitos no funcionales complementan los funcionales y fijan: perfil OWL 2 DL (para mantener decidibilidad y compatibilidad con HermiT), licencia CC-BY 4.0 para la ontología y Apache 2.0 para los scripts de validación, URIs persistentes bajo <https://w3id.org/pionera/daimo> con negociación de contenido hacia las cinco serializaciones estándar, validación SHACL accesible y, sobre todos ellos, un compromiso explícito de *reuse-first*: cada término introducido por DAIMO debe justificar en la documentación por qué DCAT, DCAT-AP, MLDCAT-AP, ODRL, PROV-O, DSP o la extensión EDC no lo cubren.

3.3. Decisiones centrales de diseño

Tres decisiones estructurales se tomaron en la fase de conceptualización y condicionan el resto del diseño.

Decisión 1: reutilización preferente. DAIMO reutiliza directamente las clases de MLDCAT-AP para todo aquello que MLDCAT-AP ya modela con precisión. `it6:MachineLearningModel` es la clase del modelo; `it6:ComputerInfrastructure` describe el entorno de ejecución; `it6:Run`, `it6:Flow` e `it6:Evaluation` capturan respectivamente la ejecución concreta, el *pipeline* asociado y la medición resultante; `dcat:Dataset`, `dcat:Distribution` y `dcat:DataService` publican el activo; `odrl:Offer` y `odrl:Agreement` expresan la política y el acuerdo. Introducir equivalentes DAIMO-nativos de cualquiera de estas clases violaría el principio de compromiso ontológico mínimo de Gruber y fracturaría el ecosistema DCAT-AP sin ganancia; una clase DAIMO-nativa sólo se justifica cuando la unión de los vocabularios reutilizados deja un hueco concreto que impide responder a al menos una pregunta de competencia.

Decisión 2: separación en tres capas complementarias. La arquitectura distingue tres capas que DAIMO conecta pero no sustituye. La capa A es el plano del espacio de datos: DSP con las extensiones EDC puntuales que DAIMO referencia. La capa B es el plano de gobernanza de plataforma: INESData (o una plataforma análoga) con sus mecanismos de registro federado y políticas operativas. La capa C es el plano del activo de IA/ML: MLDCAT-AP y los vocabularios que lo soportan. DAIMO es el perfil de integración que conecta A, B y C; no es una cuarta capa paralela que replique funcionalidad, sino un puente que hace consultable la articulación entre las tres.

Decisión 3: una clase nativa por hueco, no por tema. Dada la reutilización preferente, cada clase DAIMO-nativa debe justificar su existencia por un hueco concreto en la unión de los vocabularios reutilizados y por al menos una pregunta de competencia que no puede responderse sin ella. El criterio se aplica con rigor y produce nueve clases de primer nivel (`AIAssetOffering`, `ParticipantRole`, `ModelDeployment`, `IOContract`, `ExecutionAuthorization`, `DerivedArtifact`, `CrossParticipantProvenanceRecord`, `AuditEvidence` y `SharedEvaluationContext`) más cinco subclases de rol. Cualquier clase que no cumpla simultáneamente ambas condiciones queda descartada.

4. La ontología DAIMO

4.1. Arquitectura modular y métricas

DAIMO modela el ciclo de vida operacional de un activo de modelo de IA dentro de un espacio de datos gobernado: publicación como registro de catálogo, descubrimiento sensible a política y a calidad, invocación controlada mediante servicios tipados, trazabilidad de ejecuciones individuales y evaluación contextualizada y reproducible. El alcance se restringe deliberadamente a esta cadena operacional. Las responsabilidades ortogonales, como la modelización detallada del *pipeline* de entrenamiento, la semántica fina de políticas de privacidad o la teoría formal de la derivación PROV, ya están cubiertas por vocabularios consolidados cuya duplicación fragmentaría el ecosistema. DAIMO aporta, en cambio, los constructos necesarios para que esos vocabularios se articulen coherentemente en el caso concreto de un modelo de IA que circula entre participantes autónomos.

La ontología se estructura en tres módulos complementarios, distribuidos en archivos Turtle separados para que un consumidor pueda cargar únicamente los que necesite. El **módulo núcleo** (`daimo-core.ttl`) contiene las catorce clases y las veintinueve propiedades de objeto más ocho propiedades de datatype DAIMO-nativas, los axiomas globales de disyunción y las caracterizaciones sobre propiedades (funcionales, asimétricas, inversas) que capturan las invariantes de tipo a nivel TBox. El **módulo de alineamientos** (`alignment.ttl`) encapsula las siete declaraciones `rdfs:subClassOf` y las seis `rdfs:subPropertyOf` que conectan DAIMO con los vocabularios reutilizados, más las declaraciones mínimas de los términos externos que DAIMO referencia, cada una con un `rdfs:seeAlso` a la sección correspondiente de la especificación fuente. El **módulo SHACL** (`daimo-shapes.ttl`) aporta las dieciocho `sh:NodeShape` del perfil: nueve estructurales, tres de conformidad sobre clases reutilizadas y seis invariantes entre clases mediante `sh:SPARQLConstraint`. El módulo se declara como `owl:Ontology` independiente con su propia `owl:versionIRI`, de modo que la capa de validación se libera y referencia como un artefacto con ciclo de vida propio.

La separación en tres módulos no es accidental. El núcleo puede ser consumido por una aplicación que sólo necesite las primitivas DAIMO; el módulo de alineamientos se carga cuando se quiere que un razonador infiera las relaciones entre DAIMO y los vocabularios reutilizados; el módulo SHACL se carga cuando se quiere validar un grafo concreto contra el perfil. La Figura 1 ilustra la arquitectura modular y las relaciones con los vocabularios reutilizados.

Métricas del artefacto. Antes de recorrer las clases individuales que pueblan los tres módulos de §4.1, reportamos el tamaño y la expresividad del artefacto globalmente. La Tabla 3 extrae las métricas directamente de los archivos Turtle y de los reportes producidos por la suite de validación (§5). Los números permiten a un lector interesado estimar el peso del artefacto antes de consumirlo y compararlo con ontologías publicadas previamente en la misma familia.

[PENDIENTE: Falta una tabla complementaria al estilo OntoMetrics (plugin de Protégé) que desglose el conteo de axiomas por tipo: `SubClassOf`, `EquivalentClasses`, `DisjointClasses`, `ObjectPropertyDomain`, `ObjectPropertyRange`, `FunctionalObjectProperty`, `AsymmetricObjectProperty`, `InverseObjectProperties`, `SubDataPropertyOf`, `DataPropertyDomain`, `DataPropertyRange`, `AnnotationAssertion`, etc. Esta tabla aparece en RePlanIT §5.2 y en la documentación de la mayoría de ontologías SWJ; aporta al reviewer una medida cuantitativa de la riqueza axiomática independiente del conteo de clases y propiedades.]

4.2. Clases nativas y alineamientos

Las nueve clases nativas de primer nivel, más las cinco subclases de `ParticipantRole`, constituyen el conjunto mínimo que las tres decisiones centrales de diseño producen. La Tabla 4

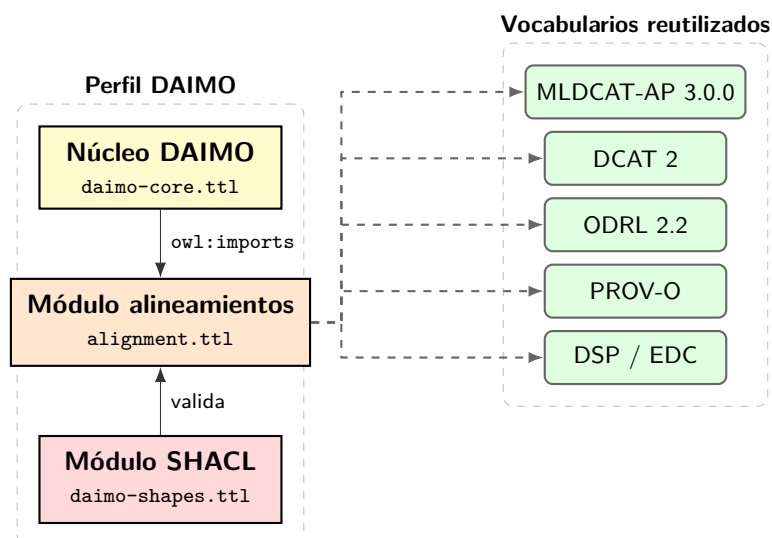


Figura 1: Arquitectura modular de DAIMO y vocabularios reutilizados. Las cajas amarilla, naranja y roja representan los tres módulos de DAIMO (núcleo, alineamientos, shapes); las verdes, los vocabularios reutilizados. Las flechas discontinuas desde el módulo de alineamientos portan las declaraciones `rdfs:subClassOf` y `rdfs:subPropertyOf`. El núcleo importa alineamientos para inferencia; el módulo SHACL valida datos contra el TBox combinado.

synetiza cada una junto con su alineamiento axiomático a los vocabularios reutilizados; las subsecciones siguientes detallan las de mayor carga semántica. La Figura 2 muestra la vista general de clases y propiedades nativas.

Las dos clases sin alineamiento externo, `IOContract` y `SharedEvaluationContext`, carecen de él deliberadamente. `IOContract` no es un `dcate:DataService` (un servicio es un recurso accesible, una URL, un endpoint, mientras que el contrato describe expectativas sintácticas y de autenticación sobre el intercambio) ni un `dct:Standard` (el contrato enumera obligaciones concretas, no refiere a una especificación externa). `SharedEvaluationContext` es una reificación metodológica, no una tarea: captura la combinación exacta bajo la que dos evaluaciones son comparables, no la tarea abstracta que dichas evaluaciones instancian. Alinearla a `owl:Thing` o `prov:Entity` añadiría ruido sin aportar semántica.

En el escenario ejecutivo de §3.1, el proveedor publica sus modelos mediante tres instancias de `AIAssetOffering` en el catálogo federado; cada oferta enlaza al modelo con `daimo:offersModel`, al publicador con `daimo:offeredBy` y a la política ODRL con `daimo:hasOfferPolicy`. El despliegue del modelo seleccionado por el consumidor municipal se modela como una `ModelDeployment` que, mediante `daimo:exposedAs`, apunta a dos `dcate:DataServices` paralelos (REST y gRPC), cada uno con su `IOContract`. La autorización que el consumidor obtiene del proveedor tras la negociación DSP se reifica como `ExecutionAuthorization` y vincula, vía `daimo:authorizesRun`, las ejecuciones concretas que se permiten. La predicción producida por cada ejecución se reifica como `DerivedArtifact` con su `AuditEvidence` asociada, y el `CrossParticipantProvenanceRecord` agrega la cadena completa a través de los `edc:ParticipantContexts` del proveedor, el consumidor y la plataforma federada.

[PENDIENTE: Falta el tratamiento por módulo. RePlanIT dedica una subsección §4.1–§4.7 a cada uno de sus siete módulos (ICTDevice, HardwareComponent, Material, CEStrategy, Agent, Indicator, Activity+Unit) con prosa que explica el hueco que cubre y el conjunto de clases que contiene; GloSIS hace lo análogo en su §3.4 Ontology Overview. Aquí las catorce clases DAIMO-nativas se condensan en una única tabla (Tabla 4) sin narrativa por agrupación. Añadir cinco subsecciones §4.3.1–§4.3.5 organizadas por rol operacional: (1) Regis-

Tabla 3: Métricas del artefacto DAIMO verificadas contra los archivos del paquete.

Dimensión	Valor
Perfil lógico declarado	OWL 2 DL
Clases nativas de primer nivel	9
Subclases nativas (ParticipantRole)	5
Total clases DAIMO-nativas	14
Propiedades de objeto (owl:ObjectProperty)	29
Propiedades de datatype (owl:DatatypeProperty)	8
Propiedades funcionales (owl:FunctionalProperty)	28
Propiedades asimétricas (owl:AsymmetricProperty)	5
Pares inversos explícitos (owl:inverseOf)	5
rdfs:subClassOf a externos	7
rdfs:subPropertyOf a externos	6
Axioma de disyunción nombrado	1
sh:NodeShape totales	18
de completitud (una por clase DAIMO)	9
de conformidad sobre reutilizadas	3
invariantes	6
sh:SPARQLConstraint	
Preguntas de competencia	23
Pitfalls OOPS! (Crítico/Importante/Menor)	0 / 0 / 2

tro y participación (AIAssetOffering + ParticipantRole con sus cinco subclases); (2) Plano de ejecución (ModelDeployment + IOContract); (3) Autorización y derivación (ExecutionAuthorization + DerivedArtifact); (4) Auditoría y procedencia (AuditEvidence + CrossParticipantProvenanceRecord); (5) Contexto de evaluación (SharedEvaluationContext). Cada subsección: 3–4 párrafos explicando el hueco que cubre, las clases que contiene, las propiedades que las articulan, y la(s) CQ(s) a las que responde.]

Defensa de los alineamientos de mayor carga. De los siete alineamientos `rdfs:subClassOf` hacia vocabularios externos, dos llevan significativamente más carga semántica que el resto y merecen defensa explícita porque las contra-lecturas son plausibles y un lector familiarizado con DCAT y ODRL las planteará inmediatamente.

AIAssetOffering $\sqsubseteq$ dcat:CatalogRecord, no dcat:Dataset. DCAT distingue con cuidado entre el recurso ofrecido (el `dcat:Dataset`, que es la cosa sobre la que hay interés) y el registro que describe dicho recurso en un catálogo concreto (el `dcat:CatalogRecord`, que es el artefacto de metadatos). DAIMO alinea `AIAssetOffering` a `dcat:CatalogRecord` porque la oferta es conceptualmente el registro de catalogación del modelo en un espacio de datos concreto, con una política y un conjunto de metadatos específicos de ese contexto, no el modelo en sí —que se reutiliza directamente como `it6:MachineLearningModel`. La contra-lectura que

Tabla 4: Clases nativas de DAIMO, alineamientos axiomáticos y preguntas de competencia que responden.

Clase	Alineamiento	Papel	CQs
daimo:AIAssetOffering	dcat:CatalogRecord	Registro de catálogo que publica un modelo como oferta gobernada en un dataspace; porta política ODRL mediante daimo:hasOfferPolicy.	R1, R2, R5, G1
daimo:ParticipantRole (+5 subclases no disjuntas)	prov:Role	Rol asumido por un participante respecto a un activo de IA dentro de un contexto de dataspace.	R5
daimo:ModelDeployment	prov:Entity	Instancia en ejecución o aloja- da del modelo, expuesta como dcat:DataService sobre una it6:ComputerInfrastructure.	E1, E3, G2
daimo:IOContract	(sin alineamiento)	Contrato mínimo accionable: formatos de E/S, método de autenticación, esquemas opcionales.	R4, E1, G2
daimo:ExecutionAuthorization	odrl:Agreement	Acuerdo ODRL derivado de una negocia- ción DSP que autoriza ejecuciones con- cretas vía daimo:authorizesRun.	E2, E5, G3
daimo:DerivedArtifact	prov:Entity, dcat:Resource	Salida gobernada y describible en catá- logo producida por una ejecución bajo au- torización.	E5, G4
daimo:CrossParticipantProvenanceRecord	prov:Entity	Agregación de actividades y entidades a través de contextos de participante para formar una narrativa de auditoría.	G4
daimo:AuditEvidence	prov:Entity	Registro de evidencia de cumplimiento con spdx:Checksum estructurado, fir- mante y marca temporal.	E4
daimo:SharedEvaluationCriteria	(sin alineamiento)	Reificación de las condiciones de compa- rabilidad: tarea, dataset, versión, proto- colo y semilla aleatoria.	V1, V2, V3

subsumiría **AIAssetOffering** a **dcat:Dataset** conlugaría dos cosas distintas: el modelo (que puede aparecer en varios catálogos con condiciones distintas) y su oferta en un catálogo específico (que es una y con una política propia). La distinción es operacionalmente relevante: una misma **it6:MachineLearningModel** puede ser referenciada por varias **AIAssetOfferings** con políticas distintas, y la propiedad **daimo:offersModel**, alineada a **foaf:primaryTopic**, captura que la oferta trata sobre el modelo sin identificarse con él.

ExecutionAuthorization $\sqsubseteq$ **odrl:Agreement**, no **prov:wasDerivedFrom odrl:Agreement**. La autorización de ejecución es el resultado específico de una negociación DSP que vincula permisos ODRL concretos a invocaciones concretas, y ODRL modela precisamente este tipo de instrumento vinculante como **odrl:Agreement** —el equivalente a un contrato suscrito entre un asignador y un asignatario, con permisos y restricciones efectivas. La contra-lectura que relacionaría la autorización con el acuerdo mediante **prov:wasDerivedFrom** trataría la autorización como un artefacto distinto del acuerdo, introduciendo una distinción que ODRL no necesita: un **Agreement** ya es la entidad vinculante. La subsunción aprovecha toda la semántica ODRL sobre permisos, prohibiciones y obligaciones sin redefinirla, y permite que herramientas ODRL existentes traten las autorizaciones DAIMO como acuerdos estándar. Lo que DAIMO añade sobre **odrl:Agreement** es la propiedad asimétrica **daimo:authorizesRun**, que vincula el acuerdo a las ejecuciones **it6:Run** que cubre: una relación que ODRL no prescribe pero que el puente al plano PROV requiere.

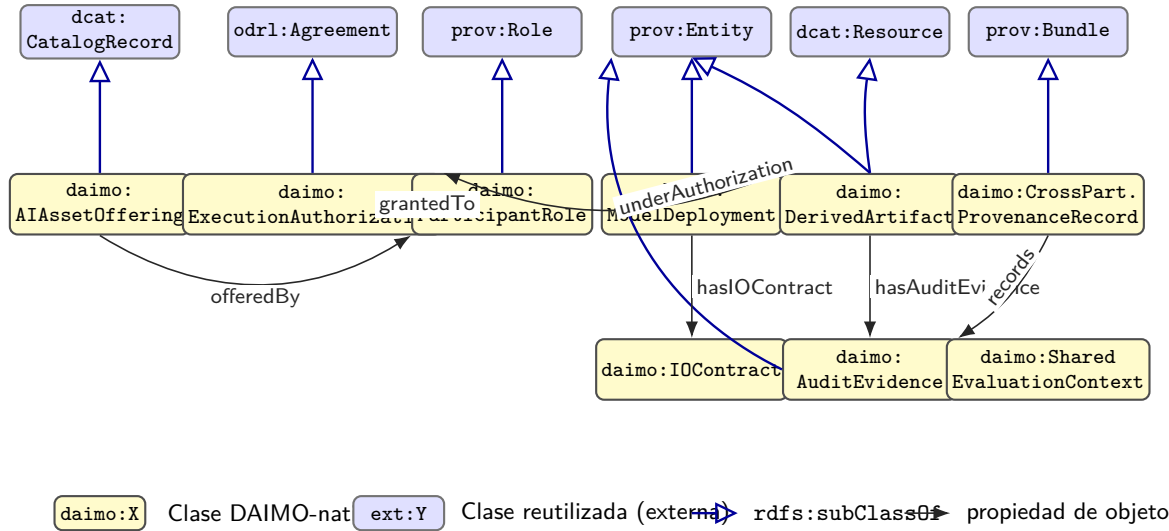


Figura 2: Vista general de las clases nativas de DAIMO (amarillo) con sus alineamientos axiomáticos hacia los vocabularios reutilizados (azul) mediante `rdfs:subClassOf` (flechas triangulares huecas), y las principales propiedades nativas que articulan el perfil (flechas etiquetadas). `DerivedArtifact` tiene dos padres externos, `prov:Entity` y `dcat:Resource`; `AuditEvidence` aparece en la fila inferior pero es también una `prov:Entity`. Las clases `IOTContract` y `SharedEvaluationContext` carecen de alineamiento externo por diseño. Las propiedades cuyos destinos son clases reutilizadas (`offersModel` hacia `it6:MachineLearningModel`, `authorizesRun` hacia `it6:Run`) aparecen en la Tabla 4 y se omiten aquí para no sobrecargar el diagrama.

[PENDIENTE: Faltan las defensas de los cinco alineamientos `rdfs:subClassOf` restantes. Un reviewer sofisticado las pedirá una por una, empezando por la más frágil. Añadir cinco párrafos cortos (uno por alineamiento) con esta cobertura:

- (1) `ParticipantRole` $\sqsubseteq$ `prov:Role` —el más frágil, porque PROV-O Role es *activity-scoped* mientras el rol DAIMO es *dataspace-scoped*; evaluar si bajar a `skos:related`;
- (2) `ModelDeployment` $\sqsubseteq$ `prov:Entity` —justificar que el despliegue es una entidad PROV, no una actividad;
- (3) `DerivedArtifact` $\sqsubseteq$ `prov:Entity`, `dcat:Resource` —defender la doble subsunción y su consistencia;
- (4) `CrossParticipantProvenanceRecord` $\sqsubseteq$ `prov:Bundle` —argumentar por qué un Bundle es el constructo adecuado frente a `prov:Collection`;
- (5) `AuditEvidence` $\sqsubseteq$ `prov:Entity` —argumentar que la evidencia es una entidad producida y no una actividad de auditar.]

4.3. Fundamentos axiomáticos

Una taxonomía por sí sola no fija el significado: sólo nombra los tipos. Detrás de la taxonomía de DAIMO hay una infraestructura axiomática que fija qué es cada clase, qué no es y cómo deben comportarse sus instancias. Tres estratos complementarios componen esta infraestructura: axiomas OWL sobre las clases y propiedades nativas, restricciones SHACL nodales sobre las instancias e invariantes SHACL-SPARQL entre clases. Los párrafos siguientes describen el primer estrato; los estratos SHACL aparecen en §?? como parte de la validación.

En el estrato OWL la disyunción entre los nueve tipos nativos de primer nivel se declara mediante el axioma nombrado `daimo:TopLevelKindsDisjointness`, instancia de `owl:AllDisjointClasses`. Convertir la disyunción en una entidad nombrada, en lugar de emitir pares anónimos `owl:disjointWith`, persigue tres beneficios. Primero, la disyunción puede referenciarse en documentación y reportes de validación. Segundo, un razonador puede citarla como causa específica cuando detecta una contradicción, facilitando el diagnóstico. Tercero, la disyunción entre las cinco subclases

de `ParticipantRole` queda deliberadamente fuera del axioma: un mismo agente jurídico puede asumir varios roles simultáneamente (por ejemplo, proveedor y operador) sin generar inconsistencia lógica, reflejando la realidad de que los roles son propiedades contextuales del agente y no clases mutuamente exclusivas.

Al núcleo de disyunciones se añaden caracterizaciones sobre las propiedades DAIMO. Veintiocho propiedades se declaran funcionales (`owl:FunctionalProperty`) sobre aquellas cuyo dominio es una clase con *shape* nodal, de modo que la cardinalidad máxima queda axiomatizada simultáneamente en la capa lógica y en la capa SHACL correspondiente. Cinco propiedades clave (`offersModel`, `deploysModel`, `authorizesRun`, `derivedFromRun` y `evidenceOf`) se marcan como asimétricas, capturando formalmente que un despliegue no es el modelo que despliega, que una autorización no coincide con la ejecución que autoriza, y así sucesivamente. Cinco pares inversos explícitos (`authorizedBy` $\leftrightarrow$ `authorizesRun`, `hasDeployment` $\leftrightarrow$ `deploysModel`, `hasDerivedArtifact` $\leftrightarrow$ `derivedFromRun`, `hasAuditEvidence` $\leftrightarrow$ `evidenceOf`, `hasOffering` $\leftrightarrow$ `offersModel`) habilitan consultas inversas nativas sin obligar al publicador del grafo a materializar ambos sentidos.

El estrato SHACL se divide a su vez en dos familias. En primer lugar, cada clase DAIMO-nativa lleva asociada una *shape* nodal de completitud que prescribe qué propiedades son obligatorias en una instancia bien formada; estas nueve *shapes* son la garantía sintáctica de que un publicador ha suministrado todo lo necesario para que las invariantes semánticas operen sobre el grafo. En segundo lugar, tres *shapes* de conformidad (`OfferInDAIMOShape`, `MachineLearningModelInDAIMOShape` y `RunInDAIMOShape`) establecen qué propiedades de clases reutilizadas son obligatorias cuando un grafo declara adherirse al perfil DAIMO. La distinción entre ambas familias es deliberada: DAIMO no sobrescribe la ontología subyacente, porque eso fragmentaría el ecosistema, sino que declara su compromiso con un subconjunto concreto de propiedades sobre las clases externas, haciendo explícito el criterio de adhesión sin modificar las definiciones originales.

Por encima de las *shapes* se sitúan las seis invariantes SHACL-SPARQL entre clases (INV-1 a INV-6) que se examinan en detalle en §5.3. Son el estrato que distingue a DAIMO de un simple perfil descriptivo: codifican reglas cuyo sentido no puede expresarse como restricción local a una sola *shape* porque involucran la coherencia entre varias clases simultáneamente. El Listado 1 muestra cómo se codifican los axiomas OWL para el caso concreto de `ExecutionAuthorization`, combinando la subclase con `odrl:Agreement`, la asimetría de `authorizesRun`, la funcionalidad de su inversa y la pertenencia al axioma de disyunción nombrado.

```
daimo:ExecutionAuthorization a owl:Class ;
  rdfs:subClassOf odrl:Agreement ;
  skos:definition "An ODRL agreement produced by a DSP contract
    negotiation that binds specific permissions to run invocations..."@en .

daimo:authorizesRun a owl:ObjectProperty , owl:AsymmetricProperty ;
  rdfs:domain daimo:ExecutionAuthorization ;
  rdfs:range it6:Run .

daimo:authorizedBy a owl:ObjectProperty , owl:FunctionalProperty ;
  owl:inverseOf daimo:authorizesRun ;
  rdfs:domain it6:Run ;
  rdfs:range daimo:ExecutionAuthorization .

daimo:TopLevelKindsDisjointness a owl:AllDisjointClasses ;
  owl:members ( daimo:AIAssetOffering
    daimo:ExecutionAuthorization
    daimo:ModelDeployment
    daimo:DerivedArtifact
    daimo:IOContract
    daimo:AuditEvidence
```

Tabla 5: Vocabularios reutilizados por DAIMO y uso concreto de cada uno.

Vocabulario	Términos reutilizados	Uso en DAIMO
MLDCAT-AP 3.0.0 (it6:)	MachineLearningModel, Task, Run, Flow, Evaluation, EvaluationMeasure, Benchmark, ComputerInfrastructure, Hardware, Library, HarmRisk, Modality, trainedOn, testedOn	Capa del activo de IA; DAIMO no redefine el modelo.
DCAT 2 (dcat:)	Catalog, Dataset, Distribution, DataService, CatalogRecord, endpointURL	Publicación y descubrimiento.
ODRL 2.2 (odrl:)	Offer, Agreement, Permission, Prohibition, hasPolicy, target, assigner, assignee	Política, oferta y acuerdo.
PROV-O (prov:)	Activity, Entity, Agent, Bundle, Role, wasGeneratedBy, wasAssociatedWith	Procedencia y roles.
SPDX (spdx:)	Checksum, algorithm, checksumValue	Integridad de evidencia de auditoría.
DSP (dspace:)	ContractOffer, ContractNegotiation, TransferProcess	Mapeos informativos <code>skos:closeMatch/skos:related</code> ; no subsunción.
EDC (edc:)	ParticipantContext	Única extensión específica de EDC referenciada.

```

daimo:CrossParticipantProvenanceRecord
daimo:SharedEvaluationContext
daimo:ParticipantRole ) .
    
```

 Listing 1: Axiomas centrales alrededor de `ExecutionAuthorization`.

Superficie de reutilización. La Tabla 5 resume los vocabularios reutilizados y el uso concreto de cada uno. Tres observaciones son importantes. La primera es que las alineaciones hacia DCAT y ODRL son con los vocabularios estándar, no con entidades internas de ninguna implementación: este punto es relevante porque en la práctica de Eclipse EDC las dos capas se confunden con frecuencia. La segunda es que DAIMO mantiene hacia DSP una relación de mapeo informativo mediante `skos:closeMatch` y `skos:related`, no de subsunción: DSP es un protocolo de mensajería, no una ontología de activos, y subsumir clases DAIMO bajo constructos DSP introduciría tipados espurios. La tercera es que de toda la extensión EDC sólo `edc:ParticipantContext` aparece explícitamente en DAIMO, referenciado por su utilidad para anclar la procedencia agregada de un participante.

Tres subpropiedades que podrían parecer naturales no se declaran, y la ausencia merece documentación explícita. `daimo:authorizesRun` no se declara subpropiedad de `prov:used`; `daimo:grantedTo` no se declara subpropiedad de `prov:qualifiedAssociation`; `daimo:evidenceOf` no se declara subpropiedad de `prov:hadActivity`. En los tres casos, el alineamiento aparentemente intuitivo produciría, bajo razonamiento RDFS u OWL-RL, que las autorizaciones se tipasen como actividades PROV, los agentes receptores como asociaciones reificadas y las evidencias como objetos de influencia. Estos tipados contradicen el modelo conceptual del puente DAIMO. La verificación de implicación (§5.2) se diseñó precisamente para detectar errores de este tipo antes de que contaminen el cierre inferido.

5. Evaluación

La evaluación sigue el patrón de cinco dimensiones de RePlanIT [9]. Primero instanciamos la ontología sobre el escenario ejecutivo como caso de estudio (§5.1); luego verificamos la ingeniería formal mediante razonamiento de consistencia, comprobación de implicación y escaneo de pitfalls (§5.2). A continuación validamos los datos a nivel de instancia frente a la capa SHACL (§5.3) y confirmamos que las veintitrés preguntas de competencia devuelven las filas esperadas (§5.4). Finalmente verificamos que los alineamientos de reutilización hacia los vocabularios externos se comportan como previsto bajo razonamiento (§5.4). Todas las dimensiones son reproducibles mediante los cuatro scripts Python del paquete público (`validate.py`, `reasoner_check.py`, `oops_check.py` y `tests/negative_test.py`).

5.1. Modelado del caso de uso

El escenario introducido en §3.1 se instancia sobre DAIMO como caso de estudio. El grafo resultante (`examples/flood-risk-scenario.ttl`) contiene 225 tripletas de datos que ejercen al menos una vez cada clase nativa de la ontología y sobre las que se ejecutan las veintitrés consultas de competencia reportadas en §5.4. Las instituciones nombradas son reales; los identificadores de modelo, valores de métrica, versiones de dataset, endpoints, acuerdos y hashes son sintéticos.

[PENDIENTE: El caso de estudio en su totalidad es pendiente y debe reemplazarse antes del envío. La sección actual es un *demostrador hipotético* construido por los autores: el modelo `flood-risk-v2` no existe, las métricas (accuracy 0,89 y 0,82) son sintéticas, la referencia a `ClimateBench-v1 v2026.1` está inventada, los endpoints (`api.inesdata.example.org`, `grpc.inesdata.example.org`) no resuelven, la ejecución `run-2026-04-20-legs` no ha ocurrido, el checksum `ex:audit-run-legs-checksum` es un valor de relleno, y ninguna de las instituciones nombradas ha confirmado el piloto. Tres caminos posibles para cerrar esta sección: (a) sustituir por un piloto real con carta de compromiso de UPM, INESData, Ayuntamiento de Leganés y CSIC, incluyendo identificadores de modelo y métricas verificables; (b) instanciar DAIMO sobre un extracto público de un catálogo MLDCAT-AP operativo (si existe alguno accesible, p. ej. del piloto SEMIC), renombrando las instituciones para que reflejen las del catálogo; (c) anonimizar a actores genéricos (“Proveedor A”, “Municipio B”, “Plataforma federada C”, “Evaluador científico D”, “Oficina de cumplimiento E”) declarando explícitamente que el caso es ilustrativo, y reemplazar `flood-risk-v2` por un identificador anónimo (`model-alpha-v2`). Todo el contenido de §5.1 —los tres párrafos que siguen y la Figura 3— se actualiza coherentemente con la opción elegida, y `examples/flood-risk-scenario.ttl` se regenera. Los 225 triples, el grafo negativo, el conteo de CQs y los reportes del razonador se reejecutan tras el cambio.]

La UPM publica, como proveedor, tres instancias de `AIAssetOffering` dentro del catálogo `ex:upm-catalog` (un `dcat:Catalog` con propiedades `dcat:record` hacia las tres ofertas). La primera, `ex:offering-flood-v2`, registra el modelo `ex:flood-risk-v2`; la segunda, `ex:offering-flood-v1-baseline`, registra la línea base `ex:flood-risk-v1`; la tercera, `ex:offering-flood-v2-ro`, registra una variante del modelo v2 con política más restrictiva. Las tres ofertas comparten la misma `it6:Task` (`ex:flood-risk-prediction`) y dominio, pero difieren en política ODRL: la oferta de baseline y la de v2 llevan una política permisiva con permiso `odrl:use`, mientras que la tercera oferta añade una `odrl:prohibition` sobre la acción de uso comercial.

El modelo `ex:flood-risk-v2` se despliega (mediante una instancia de `ModelDeployment`, `ex:deployment-flood-v2`) sobre `ex:upm-gpu-cluster`, una `it6:ComputerInfrastructure` descrita con PyTorch 2.5, CUDA 12.2 y cuatro GPUs A100. El despliegue se expone como dos `dcat:DataServices` simultáneos con sendos `IOContracts`: un endpoint REST con formato de entrada JSON, salida GeoJSON y autenticación `oauth2-bearer`, y un endpoint gRPC con pro-

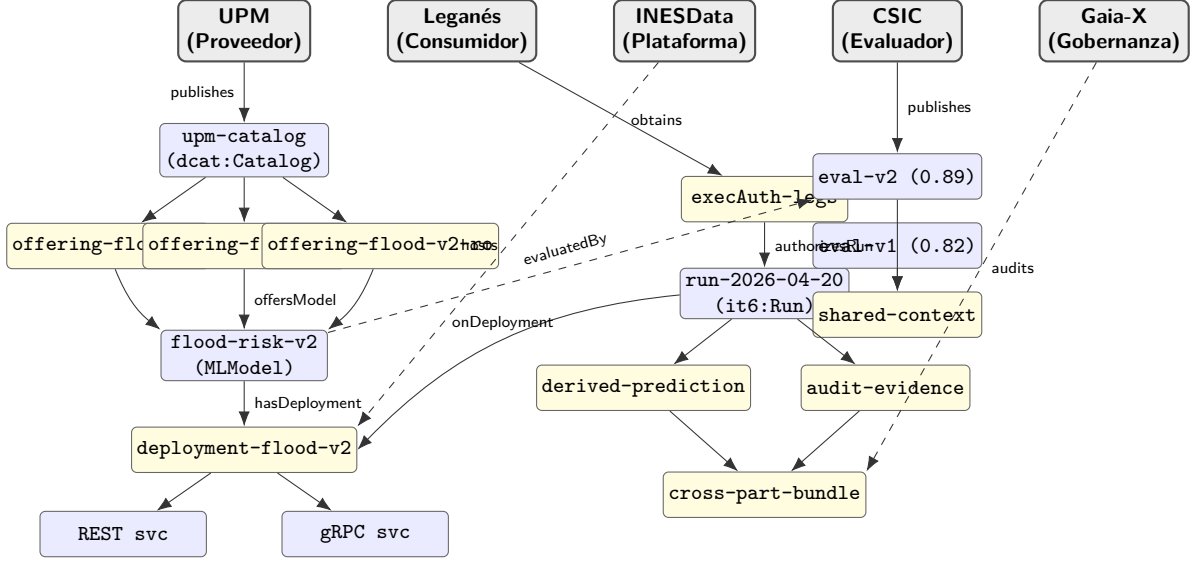


Figura 3: Escenario UPM/Leganés/INESData instanciado sobre DAIMO. Los rectángulos grises redondeados de la fila superior son los cinco participantes (uno por `edc:ParticipantContext`); las instancias sombreadas en amarillo son DAIMO-nativas (ofertas, despliegues, autorizaciones, artefactos derivados, evidencia de auditoría, bundle de procedencia cruzada, contexto compartido de evaluación); las instancias en azul claro son clases reutilizadas (catálogo, modelo, servicios, ejecuciones, evaluaciones). Las flechas sólidas denotan propiedades directas; las discontinuas denotan el rol funcional del participante sobre la instancia inferior.

tobuf y autenticación `mtls`. Este uso multi-endpoint ejercita explícitamente la decisión de modelado de tratar `daimo:exposedAs` y `daimo:hasIOContract` como propiedades no funcionales, permitiendo que un mismo despliegue anuncie varios servicios y contratos simultáneamente para cubrir distintos perfiles de autenticación.

El Ayuntamiento de Leganés, como consumidor, obtiene una instancia de `ExecutionAuthorization` ligada a un permiso `odrl:use` concreto, con fechas de vigencia explícitas. La ejecución `ex:run-2026-04-20-legs` es una `it6:Run` que `prov:wasAssociatedWith` el Ayuntamiento y que está autorizada mediante `daimo:authorizesRun`. La ejecución produce un `DerivedArtifact` (la predicción georreferenciada de riesgo de inundación por sección censal) y una `AuditEvidence` cuyo `daimo:integrityHash` apunta a un `spx:Checksum` nombrado (`ex:audit-run-legs-checksum`) con algoritmo SHA-256 y digest explícito. La promoción del checksum a nodo nombrado, en lugar de a *blank node*, permite referencias externas estables desde sistemas de archivo y logging. Una instancia de `CrossParticipantProvenanceRecord` agrupa la cadena completa abarcando tres `edc:ParticipantContexts`: UPM, Leganés e INESData. El equipo del CSIC, como evaluador, publica dos instancias de `it6:Evaluation`: la del modelo v2 con métrica `accuracy` igual a 0,89 y la de la línea base v1 con 0,82. Ambas comparten un `SharedEvaluationContext` fijado a ClimateBench-v1 versión 2026.1, protocolo *holdout* y semilla 42. La Figura 3 ilustra las instancias y sus relaciones.

5.2. Verificación formal

El razonador OWL 2 DL HermiT, invocado mediante `owlready2`, reporta que la unión del núcleo, los alineamientos y el grafo de ejemplo es lógicamente consistente y que no existen clases insatisfactibles; el tiempo de razonamiento es de 0,72 segundos. La materialización OWL-RL (biblioteca `owlrl`) deriva 1171 tripletas adicionales sobre 817 iniciales (1988 totales) en 0,27 segundos, sin producir individuos tipados como `owl:Nothing` ni subclases insatisfactibles. Ambos razonadores coinciden, con HermiT como validador de la corrección lógica y OWL-RL como materializador pragmático para las consultas que requieren inferencia por subpropiedad o

subclase.

La consistencia lógica es condición necesaria pero no suficiente para la corrección semántica de una ontología de integración. Una axiomatización puede ser perfectamente consistente y, al mismo tiempo, implicar tipados no intencionados si no declara disyunción explícita con la clase adecuada o si adopta una subpropiedad cuyo rango fuerza una subsunción indeseada. Por eso DAIMO implementa adicionalmente una *verificación de implicación* sobre el cierre OWL-RL: para cada clase nativa, el script `reasoner_check.py` enumera todas las superclases inferidas y las contrasta con una lista de superclases prohibidas. Por ejemplo, `AuditEvidence` no debe ser inferida como `prov:Activity`, porque es una cosa producida por actividades, no una actividad. El reporte cubre las catorce clases nativas (nueve de primer nivel más cinco subclases de `ParticipantRole`) y no emite ninguna advertencia. Esta comprobación es complementaria a la de consistencia y captura errores que un razonador DL acepta porque son lógicamente consistentes aunque semánticamente incorrectos.

El escáner OOPS! [30], consultado por API REST sobre la unión del núcleo y los alineamientos, reporta cero pitfalls Críticos, cero Importantes y dos Menores. El Menor P13 (“relaciones inversas no declaradas”) afecta a 34 elementos, y corresponde a propiedades para las que no existe una inversa semánticamente significativa (por ejemplo, `daimo:integrityHash`: nombrar una inversa “tiene hash” no añade capacidad de consulta). El Menor P04 (“elementos no conectados”) afecta a siete elementos que son clases externas declaradas localmente para permitir que el razonador las interprete sin exigir que el archivo las defina completamente. Ambos Menores son características habituales de perfiles de integración y no indican defectos de modelado; su aparición es precisamente el resultado de la disciplina *reuse-first* que el perfil aplica.

5.3. Validación SHACL: estructural, invariantes y banco negativo

DAIMO superpone tres familias de *shapes* SHACL, cada una respondiendo a una pregunta de validación distinta. Las *shapes* estructurales preguntan si toda instancia DAIMO-nativa porta las propiedades obligatorias de su clase. Las *shapes* de conformidad preguntan si las clases reutilizadas externas satisfacen las obligaciones adicionales que el perfil les impone. Las invariantes SPARQL preguntan si las relaciones entre varias clases siguen siendo coherentes. Juntas, las tres familias llevan la validación de lo sintáctico a lo semántico: de “¿está esta instancia bien formada?” a “¿es este grafo internamente consistente?”.

Las nueve *shapes* estructurales imponen completitud mínima por clase; el grafo positivo de 225 tripletas satisface `conforms=True`. Las tres *shapes* de conformidad añaden obligaciones operativas sobre clases reutilizadas: `OfferInDAIMOShape` exige que toda `odrl:Offer` utilizada en DAIMO declare `odrl:assigner` y `odrl:target` (a nivel de política o por permiso, mediante el conectivo `sh:or`); `MachineLearningModelInDAIMOShape` exige política ODRL, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`. Adicionalmente, `sh:in` restringe los valores de `daimo:authMethod` a un vocabulario cerrado de nueve tokens (`none`, `api-key`, `basic`, `bearer`, `oauth2-bearer`, `oauth2-client-credentials`, `mtls`, `jwt`, `dataspace-token`), y `sh:pattern` restringe `daimo:protocol` a un patrón que acepta los nombres habituales de protocolo de evaluación (`holdout`, `stratified-holdout`, `leave-one-out`, `temporal-split`, `<n>-fold-cv`, `stratified-<n>-fold-cv`, `bootstrap-<n>`).

Sobre este estrato estructural se sitúan las seis invariantes SHACL-SPARQL entre clases, codificadas mediante `sh:SPARQLConstraint`, que son el elemento diferenciador de DAIMO frente a un perfil puramente descriptivo. La invariante INV-1 garantiza la consistencia de la cadena de autorización de derivación: todo `DerivedArtifact` debe portar una propiedad `daimo:underAuthorization` cuya autorización cubra precisamente la ejecución de la que el artefacto se deriva. La INV-2 impone que el agente asociado a una ejecución mediante `prov:wasAssociatedWith` aparezca también como `daimo:grantedTo` en alguna de las autorizaciones que cubren esa ejecución, haciendo explícito que el actor registrado en PROV y el

beneficiario de la autorización ODRL deben ser la misma entidad. La INV-3 verifica la coherencia del plano de datos: el `dcat:DataService` expuesto por un `ModelDeployment` debe servir exactamente el modelo que el despliegue declara desplegar, eliminando la posibilidad de que un endpoint anuncie un modelo pero sirva otro. La INV-4 exige que el `daimo:expiresAt` de cada `ExecutionAuthorization` sea estrictamente posterior al `prov:startedAtTime` de toda ejecución que la autorización cubre, reflejando que una autorización no puede justificar ejecuciones que comenzaron tras su caducidad. La INV-5 requiere que el modelo referenciado por `daimo:offersModel` en una `AIAssetOffering` aparezca como `odrl:target` de la política asociada, a nivel de política o en alguno de sus permisos; si la política no gobierna el modelo registrado, la cadena de gobernanza está rota aunque ambos recursos existan. La INV-6 cierra el mismo triángulo por el lado del proveedor: el `daimo:offeredBy` de toda oferta debe coincidir con el `odrl:assigner` de su política, impidiendo inconsistencias entre la atribución del registro de catálogo y la autoría declarada en la oferta ODRL. El grafo positivo conforma las seis invariantes sin advertencias. El Listado 2 muestra la codificación concreta de INV-5, ilustrando el patrón `FILTER NOT EXISTS` que captura la disyunción entre política global y permiso individual.

```
daimo:OfferingPolicyTargetInvariant a sh:NodeShape ;
  sh:targetClass daimo:AIAssetOffering ;
  sh:sparql [
    a sh:SPARQLConstraint ;
    sh:message "INV-5 violation: offering's offersModel does not
              appear as odrl:target of the attached policy."@en ;
    sh:prefixes daimo:_invariantPrefixes ;
    sh:select """
      SELECT $this ?model ?policy WHERE {
        $this daimo:offersModel ?model ;
        daimo:hasOfferPolicy ?policy .
        FILTER NOT EXISTS { ?policy odrl:target ?model }
        FILTER NOT EXISTS { ?policy odrl:permission/odrl:target ?model }
      }
      """ ;
  ] .
```

Listing 2: Invariante INV-5 como `sh:SPARQLConstraint`.

Las *shapes* que conforman sobre el grafo positivo son necesarias pero no suficientes para demostrar que las restricciones son semánticamente discriminantes: una restricción trivialmente satisfacible sobre cualquier grafo pasaría la validación positiva sin aportar garantías reales. Por eso DAIMO publica un grafo complementario `tests/negative-examples.ttl` (118 tripletas) que codifica seis casos deliberadamente inválidos —`bad:INV1-artifact`, `bad:INV2-run`, `bad:INV3-deployment`, `bad:INV4-auth`, `bad:INV5-offering` y `bad:INV6-offering`—, cada uno construido para violar exactamente una invariante. El script `tests/negative_test.py` carga la ontología, los *shapes* y el grafo negativo, ejecuta SHACL y verifica dos condiciones: que la validación falle globalmente y que cada nodo focalizado esperado aparezca en el informe con el mensaje específico de la invariante violada. Las seis invariantes se disparan correctamente sobre su violación correspondiente. Esta evidencia, complementaria a la validación positiva, es la que permite afirmar con fundamento que las invariantes capturan reglas de negocio reales.

5.4. Preguntas de competencia

Las veintitrés preguntas de competencia se expresan como consultas SPARQL (`queries/queries.md`) y se ejecutan sobre el cierre OWL-RL materializado del grafo positivo. La Tabla 6 resume la cobertura por categoría, los conteos observados y las consultas que requieren razonamiento explícito; el conjunto completo de consultas aparece en el Apéndice C. Las veintitrés devuelven al menos una fila, lo que verifica que el grafo de demostración contiene la información necesaria

Tabla 6: Cobertura de las 23 preguntas de competencia. Cada CQ devuelve el número de filas indicado sobre el grafo positivo cerrado con OWL-RL. Las celdas en negrita señalan las consultas que requieren razonamiento explícito más allá de SPARQL básico.

Categoría	Consultas (filas)		N.	Inferencia requerida
Registro y publicación (R)	R1(3) R3(1) R5(2)	R2(1) R4(2)	5	R2 (cadena subPropertyOf), R5 (ruta subClassOf)
Descubrimiento y selección (D)	D1(3) D3(4) D4(1)	D2(2)	4	D1 (subtipo de tarea), D2 (coincidencia de política)
Ejecución y auditabilidad (E)	E1(4) E3(2) E5(1)	E2(1) E4(1)	5	E1 ($\text{offersModel} \sqsubseteq \text{foaf:primaryTopic}$), E2 (unión agreement-run)
Evaluación y reproducibilidad (V)	V1(1) V3(2) V5(4)	V2(1) V4(1)	5	V2, V3 (agregación)
Puente de gobernanza (G)	G1(1) G3(1) G4(1)	G2(2)	4	G3 (razonamiento subClassOf), G4 (agregación GROUP_CONCAT)

para responder a cada pregunta con las primitivas que DAIMO declara.

[PENDIENTE: Falta la tabla detallada CQ → clases → propiedades → consulta SPARQL que RePlanIT publica como Tabla 2 (apéndice, 10 páginas). Cada fila de esa tabla cubre una CQ y enumera (a) el texto de la pregunta en lenguaje natural, (b) las clases DAIMO involucradas, (c) las propiedades recorridas, (d) el fragmento SPARQL completo, (e) el conteo de filas devueltas y (f) un ejemplo de fila. El apéndice actual sólo redirige a queries/queries.md; un reviewer pedirá la tabla expandida dentro del paper para poder inspeccionarla sin salir del PDF.]

Dos resultados conviene señalar. Las consultas CQ-R2 y CQ-E1 dependen de la relación subPropertyOf entre daimo:offersModel y foaf:primaryTopic y no devuelven filas sobre un motor SPARQL puro sin razonamiento. La dependencia es documentada, no un defecto: alinear offersModel bajo foaf:primaryTopic hace las ofertas descubribles mediante la propiedad DCAT estándar, beneficio que sólo se materializa bajo inferencia. Un consumidor que necesite ejecutar esas consultas sobre un endpoint puro puede materializar el cierre previamente o reescribir las consultas enumerando ambas propiedades. El segundo resultado es que CQ-G2 devuelve dos filas —una por endpoint del despliegue multi-protocolo REST y gRPC—, lo que confirma el modelado de múltiples servicios por despliegue como patrón legítimo y directamente consultable desde SPARQL.

Reutilización verificada. La reutilización se verifica directamente sobre el módulo de alineamientos. Siete rdfs:subClassOf y seis rdfs:subPropertyOf, listados en la Tabla 4 y visualizados en la Figura 2, conectan DAIMO con sus cinco vocabularios parentales (DCAT, MLDCAT-AP, ODRL, PROV-O y FOAF). La verificación opera en dos dimensiones.

La primera es *estructural*: toda clase nativa que podría haberse definido aisladamente se sitúa bajo un padre externo siempre que dicho padre exista, y toda propiedad nueva extiende una propiedad reutilizada siempre que ésta ya cargue el significado previsto. No se introduce ninguna clase por razones estilísticas, y las clases IOContract y SharedEvaluationContext quedan sin padre externo sólo porque ningún padre con semántica veraz existe (§4.2).

La segunda es *semántica*: los alineamientos no deben producir inferencias no intencionadas. La verificación de implicación de §5.2 inspecciona el cierre OWL-RL de cada alineamiento y confirma que ninguno produce una superclase semánticamente incorrecta sobre ninguna clase nativa. El razonador reporta cero advertencias sobre las catorce clases. Tres alineamientos de subpropiedad hacia PROV-O se excluyeron deliberadamente del módulo por el mismo motivo:

cada uno habría producido un tipado prohibido bajo cierre RDFS (§4.3).

6. Discusión

DAIMO articula publicación, política, ejecución, procedencia y evaluación en un único perfil de integración que sostiene el intercambio gobernado de modelos a través de fronteras organizativas. Esta posición la diferencia tanto de ontologías de dominio publicadas recientemente —RePlanIT [9], GloSIS [10] o la ontología de mantenimiento [11]— como de las ontologías previas del ciclo de vida de aprendizaje automático. Con las primeras comparte la ética de ingeniería ontológica —reutilización disciplinada, implementación ejecutable y evaluación contra preguntas de competencia— pero la aplica a la articulación entre catálogo de IA y plano de datos. Con las segundas comparte el dominio de aplicación pero suma la capa de gobernanza que ellas no abordan. Tres rasgos de diseño diferencian la ontología. El primero es la verificación de implicación sobre el cierre OWL-RL, que complementa al razonamiento de consistencia enumerando las superclases inferidas de cada clase nativa y detectando alineamientos semánticamente incorrectos que un razonador DL acepta sin contradicción. El segundo son las invariantes SHACL-SPARQL entre clases, acompañadas de un banco de pruebas negativas que confirma su carácter discriminante: cada invariante se dispara sobre un caso diseñado para violarla, no sólo sobre el grafo positivo. El tercero es la orientación *reuse-first*, encarnada en alineamientos axiomáticos explícitos y en la decisión documentada de no declarar tres subpropiedades hacia PROV-O cuya adopción habría producido tipados no intencionados sobre clases de artefacto.

Las limitaciones del perfil se agrupan en tres líneas argumentales. La primera concierne al *nivel de detalle semántico* de varias extensiones DAIMO. El `IOContract` captura el formato de entrada/salida y el método de autenticación; este último está restringido por `sh:in` a un vocabulario cerrado. No cubre, en cambio, la compatibilidad semántica fina entre los esquemas del modelo y del consumidor: esas dimensiones se delegan a `it6:Flow`, como hacen también Kipoi [20] y TFX [21] para preprocesamiento y *pipelines*. El `SharedEvaluationContext` reifica la combinación tarea-dataset-versión-protocolo-semilla suficiente para comparaciones homogéneas sobre una métrica única, pero no cubre protocolos heterogéneos de *benchmarking* ni conjuntos compuestos de métricas. El modelado de roles conflata el tipo (`ModelProvider`) con la asignación —la tupla agente-rol-contexto-alcance—, confluencia que OntoClean recomienda separar. MLDCAT-AP adopta la misma pragmática en `hasRegisteredUser`, lo que sitúa la contextualización fina en el horizonte del refinamiento futuro, no en el del error de diseño.

La segunda línea concierne a los *compromisos de integración* y a los efectos que tienen sobre consumidores que combinen DAIMO con vocabularios adyacentes. La propiedad `daimo:records` declara `rdfs:range prov:Activity`: bajo razonamiento RDFS, las evaluaciones MLDCAT-AP agrupadas dentro de un `CrossParticipantProvenanceRecord` son inferidas como actividades PROV aunque MLDCAT-AP no declare esa subsunción. El efecto es inocuo para consultas de auditoría, pero un consumidor que integre DAIMO, MLDCAT-AP y PROV-O puede obtener tipados sorprendentes sobre `it6:Evaluation`. En la misma línea, DAIMO no se ancla a una ontología de nivel superior (BFO, GFO, DOLCE o UFO): la ausencia responde a que un perfil de integración no es una ontología de referencia. Un trabajo futuro coherente sería acomodar la taxonomía a una de esas ontologías superiores sin renunciar a la reutilización directa de los vocabularios de dominio que motivan DAIMO.

La tercera línea concierne a las *condiciones de reproducibilidad y validación externa*. Los resultados numéricos reportados dependen de un perfil de razonamiento concreto: el modo `inference=rdfs` en `pyshacl`, necesario para que las subpropiedades implicadas se materialicen antes de evaluar los *shapes* de conformidad, y el cierre OWL-RL materializado antes de las consultas de competencia. Esta dependencia está documentada pero es una condición que un consumidor debe conocer para reproducir los resultados exactos. La validación con expertos del dominio no forma parte de la evidencia reportada aquí; un estudio cualitativo con tres

perfiles (ingeniero EDC/INESData, mantenedor MLDCAT-AP/SEMIC y practicante MLOps) constituye la siguiente fase de trabajo. **[PENDIENTE: Ejecutar el estudio con expertos: reclutamiento, protocolo de entrevista semiestructurada, análisis cualitativo y publicación de resultados.]**

Conviene delimitar la relación entre DAIMO y MLDCAT-AP para disipar toda preocupación de duplicidad. El modelo propiamente dicho se reutiliza directamente mediante `it6:MachineLearningModel`, sin redefinición. La aportación de DAIMO se concentra en la articulación entre catálogo y plano de datos: las cuatro preguntas de competencia de la categoría de gobernanza ejercitan exclusivamente clases DAIMO-nativas y no pueden responderse sólo con MLDCAT-AP, pues requieren `AIAssetOffering` (CQ-G1), `ModelDeployment` (CQ-G2), `ExecutionAuthorization` (CQ-G3) y `CrossParticipantProvenanceRecord` (CQ-G4). Los dos perfiles son complementarios.

Más allá del artefacto presente, DAIMO ilustra un patrón que probablemente se repetirá allí donde participantes autónomos necesiten intercambiar recursos de alto valor sujetos a política. El patrón tiene tres elementos recurrentes: un registro de oferta reificado sobre el propio recurso, una autorización reificada sobre el permiso, y un bundle de procedencia que cruza contextos administrativos. Conjuntos de datos genómicos, herramientas de apoyo a la decisión clínica, gemelos digitales y modelos industriales presentan cadenas análogas de publicación-invocación-auditoría. DAIMO es específica de IA en lo que reutiliza (MLDCAT-AP), pero agnóstica al dominio en los constructos que añade; confiamos en que esos constructos sirvan como punto de partida para perfiles hermanos en dominios adyacentes.

[PENDIENTE: Falta un párrafo de adopción e integración con iniciativas reales. RePlanIT §5.4 discute su encaje con FEDeRATED, BattInfo y Catena-X (la Digital Product Passport Ecosystem); GloSIS §4 describe su instanciación sobre LUCAS, SRDB y WoSIS. DAIMO debería discutir explícitamente en esta sección su encaje con (a) el dossier SEMIC/MLDCAT-AP (para alinear la evolución del perfil europeo de catalogación de IA), (b) el conector Eclipse EDC y los despliegues INESData (para validar el puente al plano DSP), (c) Gaia-X y su Trust Framework (para la capa de cumplimiento), y (d) los requisitos operativos del AI Act Anexo IV (para la trazabilidad exigida a sistemas de alto riesgo). Este párrafo responde a la pregunta que un reviewer hace siempre: ¿dónde se usaría DAIMO de verdad?]

7. Disponibilidad y sostenibilidad

DAIMO se distribuye como paquete público bajo licencia CC-BY 4.0 para la ontología y su documentación, y Apache 2.0 para los scripts de validación. La Tabla 7 resume los elementos entregables y sus ubicaciones. El procedimiento operativo de liberación se documenta en `DEPLOYMENT.md`, de modo que cualquier mantenedor futuro puede ejecutarlo sin conocimiento implícito adicional. Los elementos en rojo indican acciones de publicación pendientes.

El flujo de reproducibilidad no requiere el despliegue previo de la URL persistente: cualquier usuario con Python 3.12 y las dependencias declaradas (`rdflib`, `pyshacl`, `owlrl`, `owlready2`) puede ejecutar los cuatro scripts localmente y reproducir íntegramente los reportes de `reports/`. La URL persistente y el DOI son requisitos FAIR adicionales que complementan pero no condicionan la reproducibilidad. La cabecera de la ontología declara `dct:conformsTo` hacia OWL 2 y utiliza `rdfs:seeAlso` para enlazar los tres módulos y la documentación HTML, de modo que un consumidor puede recorrer la cadena identificador-documento sin cargar el paquete completo.

La sostenibilidad del artefacto se estructura según la Fase 4 de LOT. El fichero `CONTRIBUTING.md` documenta el flujo de triaje de *issues*, el ciclo de pull requests y la política de versionado según SemVer: las versiones *patch* se reservan para correcciones de definición textual o de documentación, las *minor* para nuevas clases o propiedades compatibles y las *major* para cambios que alteren alineamientos o invariantes. La política de deprecación impone al menos una versión *minor* de coexistencia antes de retirar un término, con `owl:deprecated` y `skos:closeMatch`

Tabla 7: Disponibilidad del artefacto DAIMO como paquete público reproducible.

Elemento	Ubicación
Namespace persistente	https://w3id.org/pionera/daimo — [PENDIENTE: Ejecutar merge del PR sobre perma-id/w3id.org para que el namespace resuelva por content negotiation.]
Módulos versionados	Tres ontologías independientes (núcleo, <i>shapes</i> , alineamientos) bajo https://w3id.org/pionera/daimo/ con <code>owl:versionIRI</code> y <code>owl:priorVersion</code> explícitos
Repositorio público	[PENDIENTE: Crear repositorio GitHub pionera/daimo ejecutando DEPLOYMENT.md e incluir aquí la URL definitiva.]
Archivado versionado (DOI)	[PENDIENTE: Archivar la primera <i>release</i> en Zenodo mediante la integración GitHub → Zenodo e incluir aquí el DOI resultante.]
Serializaciones	Turtle, RDF/XML, JSON-LD, N-Triples (pre-generadas por WIDOCO)
Documentación HTML	WIDOCO 1.4.25 con WebVOWL integrado; negociación de contenido vía <code>.htaccess</code> bajo <code>w3id-redirect/</code>
Referencia humana-legible	ONTOLOGY-REFERENCE.md (clase por clase con <code>skos:definition</code> , <code>skos:example</code> , criterios de identidad y etiquetas OntoClean) y VALIDATION-MATRIX.md
Metadatos de <i>release</i>	CITATION.cff, <code>.zenodo.json</code> y CHANGELOG.md
Paquete de validación	<code>validate.py</code> , <code>reasoner_check.py</code> , <code>oops_check.py</code> , <code>tests/negative_test.py</code>
Gobernanza de mantenimiento	CONTRIBUTING.md (Fase 4 LOT: triaje de <i>issues</i> , pull requests, SemVer, política de deprecación)

hacia el término de reemplazo.

El paquete satisface los cuatro principios FAIR. Es *findable* a través del namespace persistente `w3id.org`, con IRIs por versión y metadatos Dublin Core completos. Es *accessible* mediante negociación de contenido y cinco serializaciones pre-generadas bajo licencia CC-BY 4.0 abierta. Es *interoperable* por su compromiso con vocabularios W3C estándar (DCAT, ODRL, PROV-O, SKOS, FOAF, DCMI) mediante alineamientos axiomáticos explícitos en perfil OWL 2 DL. Es *reusable* gracias a definiciones y ejemplos SKOS por término, metadatos de procedencia declarados, versionado SemVer y las *shapes* SHACL como contrato de datos.

8. Conclusiones

DAIMO conecta MLDCAT-AP, DCAT, ODRL, PROV-O y el protocolo DSP para convertir los modelos de IA en activos gobernables de primera clase dentro de un espacio de datos. Su contribución específica es la capa de gobernanza ausente entre el catálogo de IA y el plano de datos —registro con política ODRL coherente, autorización de ejecución anclada a un acuerdo, despliegue como puente tipado entre el modelo y su servicio, y procedencia agregada entre contextos administrativos—, capa que los vocabularios reutilizados aisladamente no proporcionan y que cuatro de las veintitrés preguntas de competencia (la categoría G) permiten verificar de forma concreta. El caso de estudio multi-participante ilustra que la ontología puede instanciarse coherentemente sobre un escenario verosímil y responder las cinco familias de consultas operativas que un espacio de datos gobernado plantea. Las líneas de trabajo futuro se derivan directamente de las limitaciones acotadas: instanciación contra catálogos MLDCAT-AP reales, evaluación cualitativa con expertos del dominio, refinamiento del `IOContract` hacia compatibilidad semántica de esquemas, separación rol-tipo/rol-asignación siguiendo OntoClean y extensión del `SharedEvaluationContext` a protocolos heterogéneos de *benchmarking*. El artefacto se distribuye como paquete público con namespace persistente, documentación WIDOCO, capa SHACL independiente y suite de validación reproducible, preparado para ser adoptado, extendido o criticado por la comunidad con la misma disciplina metodológica con la que ha sido

construido.

Referencias

- [1] Franklin, M., Halevy, A. and Maier, D. (2005) From Databases to Dataspaces: A New Abstraction for Information Management. *SIGMOD Record*, 34(4):27–33.
- [2] Albertoni, A. et al. (2020) *Data Catalog Vocabulary (DCAT) — Version 2*. W3C Recommendation.
- [3] Lebo, P., Sahoo, S. and McGuinness, D. (eds.) (2013) *PROV-O: The PROV Ontology*. W3C Recommendation.
- [4] Iannella, R., Guth, S. and Steyskal, R. (eds.) (2018) *ODRL Information Model 2.2*. W3C Recommendation.
- [5] Knublauch, H. and Kontokostas, D. (eds.) (2017) *Shapes Constraint Language (SHACL)*. W3C Recommendation.
- [6] Mitchell, M. et al. (2019) Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*.
- [7] Arnold, M. et al. (2019) FactSheets: Increasing Trust in AI Services through Supplier’s Declarations of Conformity. *IBM Research Report*.
- [8] Werbrouck, J. et al. (2024) ConSolid: A federated ecosystem for heterogeneous multi-stakeholder projects. *Semantic Web*, 15:429–460.
- [9] Kurteva, A. et al. (in press) RePlanIT Ontology for Digital Product Passports of ICT: Laptops and Data Servers. *Semantic Web*.
- [10] Palma, R. et al. (in press) GloSIS: The Global Soil Information System Web Ontology. *Semantic Web*.
- [11] Woods, C. et al. (in press) An ontology for maintenance activities and its application to data quality. *Semantic Web*.
- [12] Penteado, B.E., Maldonado, J.C. and Isotani, S. (2023) Methodologies for publishing linked open government data on the Web. *Semantic Web*, 14:585–610.
- [13] Bareedu, Y.S. et al. (2024) Deriving semantic validation rules from industrial standards: An OPC UA study. *Semantic Web*, 15:517–554.
- [14] Ferranti, N. et al. (in press) Formalizing and Validating Wikidata’s Property Constraints using SHACL and SPARQL. *Semantic Web*.
- [15] Pandit, H.J. and Esteves, B. (in press) Enhancing Data Use Ontology (DUO) for Health-Data Sharing by Extending it with ODRL and DPV. *Semantic Web*.
- [16] Robaldo, L. and Batsakis, S. (in press) On the interplay between validation and inference in SHACL. *Semantic Web*.
- [17] Otto, B. et al. (2019) Reference architecture model for international data spaces. *International Journal of Information Management*, 45:182–199.
- [18] Cappiello, C., Gal, A., Jarke, M., Rehof, J. and Yang, Y. (2022) Data spaces: Design, deployment, and future directions. *ACM Computing Surveys*, 55(2):Article 46.

- [19] Gebru, T. et al. (2021) Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.
- [20] Avsec, Z. et al. (2019) The Kipoi repository accelerates community exchange and reuse of predictive models for genomics. *Nature Biotechnology*, 37.
- [21] Baylor, D. et al. (2017) TFX: A TensorFlow-based production-scale machine learning platform. *KDD*.
- [22] Soldatova, L.N. and King, R.D. (2006) An ontology of scientific experiments. *Journal of the Royal Society Interface*, 3(11).
- [23] Correa Publio, G. et al. (2018) ML-Schema: Exposing the semantics of machine learning with schemas and ontologies.
- [24] Keet, C.M., Fernández, J.D. and Panov, P. (2016) MEX vocabulary: A lightweight interchange format for machine learning experiments. *ISWC*.
- [25] Panov, P., Dzeroski, S. and Soldatova, L.N. (2008) OntoDM: An ontology of data mining. *ICDMW*.
- [26] Keet, C.M. et al. (2015) The data mining optimization ontology. *Web Semantics*, 32.
- [27] Souza, R. et al. (2019) Provenance data in the machine learning lifecycle in computational science and engineering. *WORKS*.
- [28] Schmidt, M. et al. (2021) FAIRness along the machine learning lifecycle using Dataverse in combination with MLflow. *ICDE Workshops*.
- [29] Poveda-Villalón, M., Fernández-Izquierdo, A., Fernández-López, M. and García-Castro, R. (2022) LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755.
- [30] Poveda-Villalón, M., Gómez-Pérez, A. and Suárez-Figueroa, M.C. (2014) OOPS! (Ontology Pitfall Scanner!). *International Journal on Semantic Web and Information Systems*, 10(2):7–34.
- [31] SEMIC (2025) *MLDCAT-AP 3.0.0: Machine Learning Data Catalogue Application Profile*. European Commission.
- [32] International Data Spaces Association (2024) *Dataspace Protocol (DSP)*. W3C Community Draft.
- [33] Eclipse Foundation. *Eclipse Dataspace Components (EDC)*, control-plane documentation.
- [34] Garijo, D. (2017) WIDOCO: A Wizard for Documenting Ontologies. *ISWC*.
- [35] European Union (2024) *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Official Journal of the European Union.

A. Preguntas de competencia

R — Registro y publicación (5)

1. CQ-R1: ¿Qué modelos ha publicado un proveedor dado como activos de catálogo gobernados?
2. CQ-R2: Dado un modelo, ¿qué oferta lo registró, por qué publicador y en qué fecha? (requiere cadena `subPropertyOf`)

3. CQ-R3: ¿Qué licencia y política de uso aplican a un modelo publicado?
4. CQ-R4: ¿Qué contrato de entrada/salida se declara para la interfaz de invocación?
5. CQ-R5: Para cada oferta, ¿qué subclase concreta de `ParticipantRole` tiene el agente proveedor? (requiere `subClassOf+`)

D — Descubrimiento y selección (4)

1. CQ-D1: ¿Qué modelos resuelven una tarea dada en un dominio dado?
2. CQ-D2: ¿Qué modelos son utilizables bajo un patrón de política concreto (por ejemplo uso no comercial)?
3. CQ-D3: ¿Qué modelos exponen un endpoint y qué método de autenticación requiere cada uno?
4. CQ-D4: ¿Qué modelos alcanzan un umbral mínimo de métrica bajo un contexto de evaluación dado?

E — Ejecución y auditabilidad (5)

1. CQ-E1: Dada una oferta, ¿qué endpoint, método de autenticación y contrato de entrada/salida aplican al modelo subyacente? (requiere entailment)
2. CQ-E2: Para un despliegue dado, ¿qué ejecuciones, por qué agentes y bajo qué autorización?
3. CQ-E3: Para una ejecución dada, ¿qué implementación, algoritmo, *flow* e infraestructura?
4. CQ-E4: ¿Qué evidencia de auditoría (hash, firmante, marca temporal) respalda una ejecución?
5. CQ-E5: ¿Qué artefactos derivados produjo una ejecución y bajo qué autorización?

V — Evaluación y reproducibilidad (5)

1. CQ-V1: ¿Qué contexto compartido (dataset, versión, protocolo, semilla) usa una evaluación?
2. CQ-V2: Bajo un contexto compartido, ¿qué modelo alcanza el mayor valor de métrica?
3. CQ-V3: ¿Cómo se ordenan dos o más modelos bajo el mismo contexto y métrica?
4. CQ-V4: Para un modelo dado, ¿sobre qué benchmark suites ha sido evaluado?
5. CQ-V5: ¿Qué artefactos de reproducibilidad (flow, cuaderno, tabla de resultados, checksum) respaldan una evaluación?

G — Puente de gobernanza (4, DAIMO-nativas)

1. CQ-G1: ¿Qué ofertas del catálogo federado incluyen un modelo dado?
2. CQ-G2: ¿Qué despliegues sirven un modelo dado, sobre qué infraestructura y con qué contrato de entrada/salida?
3. CQ-G3: ¿Qué autorización, y el acuerdo del que deriva, permitió una ejecución específica?
4. CQ-G4: A través de contextos de participante, ¿cuál es el bundle de procedencia completo para un artefacto derivado?

B. Shapes SHACL

La capa SHACL de DAIMO contiene nueve *shapes* nodales de completitud, tres *shapes* de conformidad sobre clases reutilizadas y seis invariantes SHACL-SPARQL entre clases. El módulo `shapes/daimo-shapes.ttl` se declara a su vez como `owl:Ontology` independiente con `owl:versionIRI` bajo <https://w3id.org/pionera/daimo/shapes/>.

- **Shapes de completitud (9):** `AIAssetOfferingShape`, `ParticipantRoleShape`, `ModelDeploymentShape`, `IOContractShape`, `ExecutionAuthorizationShape`, `DerivedArtifactShape`, `SharedEvaluationContextShape`, `AuditEvidenceShape` y `CrossParticipantProvenanceRecordShape`.
- **Shapes de conformidad sobre clases reutilizadas (3):** `OfferInDAIMOShape` exige `odrl:assigner` y `odrl:target` sobre toda `odrl:Offer`; `MachineLearningModelInDAIMOShape` exige política, título e identificador sobre `it6:MachineLearningModel`; `RunInDAIMOShape` exige *flow*, algoritmo, agente asociado y marca temporal sobre `it6:Run`.
- **Invariantes cruzadas (6):** `DerivationAuthorizationInvariant` (INV-1), `RunAgentAuthorizationInvariant` (INV-2), `DeploymentServiceInvariant` (INV-3), `AuthorizationTemporalInvariant` (INV-4), `OfferingPolicyTargetInvariant` (INV-5) y `OfferingAssignerInvariant` (INV-6); todas como `sh:SPARQLConstraint`.

C. Consultas SPARQL

Las veintitrés consultas SPARQL correspondientes a las preguntas del Apéndice A están publicadas en `queries/queries.md`. El validador ejecuta cada consulta sobre el cierre OWL-RL del grafo unión ontología más ejemplo y verifica que cada una devuelve al menos una fila. Los conteos exactos aparecen en la Tabla 6. CQ-G2 devuelve dos filas —una por cada endpoint del despliegue multi-protocolo REST y gRPC—, lo que confirma que el modelado de múltiples servicios por despliegue es un patrón legítimo y directamente observable desde SPARQL.

D. Glosario de abreviaturas

[PENDIENTE: Falta el glosario de abreviaturas. GloSIS cierra el artículo con una sección *Glossary* que enumera todos los acrónimos y abreviaturas usados. Construir aquí una tabla con al menos: AI Act (Regulation EU 2024/1689), CQ (Competency Question), DAIMO (Data-space AI Model Ontology), DCAT (Data Catalog Vocabulary), DCAT-AP (DCAT Application Profile), DL (Description Logic), DOI (Digital Object Identifier), DPV (Data Privacy Vocabulary), DSP (Data-space Protocol), EDC (Eclipse Dataspace Components), FAIR (Findable, Accessible, Interoperable, Reusable), LOT (Linked Open Terms), MLDCAT-AP (Machine Learning Data Catalogue Application Profile), ODRL (Open Digital Rights Language), OOPS! (Ontology Pitfall Scanner), OWL (Web Ontology Language), PROV-O (PROV Ontology), RDFS (RDF Schema), SHACL (Shapes Constraint Language), SKOS (Simple Knowledge Organization System), SPARQL (SPARQL Protocol and RDF Query Language), SWJ (Semantic Web Journal), TTL (Turtle), WIDOCO (Wizard for Documenting Ontologies).]

E. Convenciones de URI

DAIMO utiliza el namespace canónico <https://w3id.org/pionera/daimo>, con identificadores de términos en la forma <https://w3id.org/pionera/daimo#<Término>> e IRIs versionadas bajo <https://w3id.org/pionera/daimo/<versión>>, enlazadas entre sí mediante `owl:priorVersion`.

Los módulos satélite son ontologías independientes con IRIs versionadas propias: la capa SHACL bajo <https://w3id.org/pionera/daimo/shapes/> y el módulo de alineamientos bajo <https://w3id.org/pionera/daimo/align> (este último declarando `owl:imports` sobre la ontología base). El grafo de ejemplo utiliza el namespace local <https://example.org/daimo-scenario/>, claramente separado del espacio canónico. La negociación de contenido resuelve el mismo identificador a HTML, Turtle, RDF/XML, JSON-LD o N-Triples en función del `Accept` header; la configuración `.htaccess` correspondiente está preparada en `w3id-redirect/.htaccess` a la espera de su integración en `perma-id/w3id.org`.